

Nucamp Solo AI — Submission documentation bundle

Generated 2026-05-02 02:57 UTC · nucamp_soloai repository

Automation snapshot (evidence)

- **Vitest:** 20 tests passed (7 files) — `run npm run test:unit -- --run`
- **Playwright:** 12 tests passed — `run npm run test:e2e` (requires dev server / Chrome)
- **Tailwind / DaisyUI:** `daisyui` added as devDependency so `tailwind.config.js` resolves under `@tailwindcss/vite`.
- **Typecheck:** `npm run check` (run locally before deploy)

Source: docs/code-review-billing-analytics-criteria.txt

PROMPT (copy for Claude Code or similar)

Review the nucamp_soloai SvelteKit codebase against these criteria and cite file paths: (1) Pricing page shows all subscription tiers with redirect to Stripe or Lemon Squeezy checkout. (2) Payment confirmation visible after successful payment (app or dashboard). (3) Persisted fields for stripeCustomerId, subscriptionStatus, and webhook processing logs. (4) Whether Stripe webhook handling supports Stripe CLI forward-and-verify flows. (5) Google Tag Manager / GA4 instrumentation vs dashboard proof. (6) Hotjar integration in code vs Hotjar dashboard "connected site." Note gaps between code and operational verification.

CODEBASE REVIEW — Billing, Webhooks, and Analytics Criteria

Project: nucamp_soloai (SvelteKit) Review date: 2026-05-01 Scope: Static code and configuration review. Items involving external dashboards (GA4, GTM UI, Hotjar UI) or live Stripe CLI output are marked as OPERATIONAL VERIFICATION REQUIRED.

1. PRICING PAGE — All subscription tiers + redirect to Stripe or Lemon Squeezy
VERDICT: PASS (with documented behavior)
Evidence:
• Server load defines three tiers (basic, pro, team) with Stripe price IDs and/or Lemon variant IDs from environment variables. File: src/routes/pricing/+page.server.ts Plans: basic / pro / team with envPrice('STRIPE_PRICE_*') and envVariant('LEMON_VARIANT_*'). • Checkout processor is locale-based: English (base locale) uses Stripe; non-English Paraglide locales use Lemon Squeezy. File: src/lib/server/billing/payment-provider.ts Function: paymentProcessorForLocale(locale) • Client calls POST /api/stripe/checkout with { priceld } and redirects via window.location.href to returned Checkout URL when processor is stripe. Calls POST /api/lemonsqueezy/checkout with { tier } when processor is lemon. File: src/routes/pricing/+page.svelte (startCheckout)
Caveats:
• Tiers only render if the corresponding env is set (e.g. plan.priceld for Stripe branch, plan.lemonVariantId for Lemon branch). If env vars are missing, the UI shows configuration hints instead of all three cards. • Stripe and Lemon are not offered on the same pricing view simultaneously; one processor applies per request locale.
2. PAYMENT CONFIRMATION — Success visible in app or dashboard
VERDICT: PARTIAL
Evidence:
• Stripe success URL and cancel URL are configured in validated env (STRIPE_SUCCESS_URL, STRIPE_CANCEL_URL). Lemon equivalents: LEMON_SQUEEZY_CHECKOUT_SUCCESS_URL, LEMON_SQUEEZY_CHECKOUT_CANCEL_URL. File: src/lib/server/env.ts (ValidatedEnv) • After checkout, subscription state is intended to update via webhooks writing to the billing store; the Account page reads billing and displays provider, status, tier, renewal/end date, price or variant id, and portal actions (Stripe customer portal when configured; Lemon portal URL when present). Files: src/routes/account/+page.server.ts, src/routes/account/+page.svelte
Gaps:
• No dedicated "Payment successful" route or query-parameter handler (e.g. ?session_id=) was found that shows a one-time confirmation banner. Confirmation is indirect: user lands on account (or configured success URL) and sees subscription section once webhooks have updated data.
Recommendation for reviewers: complete a test payment and confirm Account subscription block updates within webhook latency.
3. PAYMENT DATABASE FIELDS — stripeCustomerId, subscriptionStatus, webhook logs
VERDICT: PASS for fields; CLARIFICATION for "database"
Evidence:
• Persistent billing shape includes stripeCustomerId, subscriptionStatus, stripeSubscriptionId, priceld, subscriptionTier, subscriptionEndDate, Lemon fields, subscriptionProvider, updatedAt, etc. File: src/lib/server/stripe/billing-store.ts (type UserBilling) • Webhook idempotency / audit of processed events:

- Stripe: data/stripe-webhook-events.json (event id → event type string)
 - Lemon: data/lemon-webhook-events.json Functions: wasWebhookProcessed, markWebhookProcessed, wasLemonWebhookProcessed, markLemonWebhookProcessed File: src/lib/server/stripe/billing-store.ts
- Primary billing document: data/stripe-billing.json (keyed by user id)

Clarification:

- Storage is JSON files on disk under ./data/, not a relational SQL schema. If the criterion strictly requires RDBMS columns, this is a gap; if “persisted subscription state” is acceptable, it is implemented.

4. STRIPE WEBHOOK EVENTS — Stripe CLI logs showing successful processing

VERDICT: NOT VERIFIABLE FROM REPOSITORY (implementation present)

Evidence:

- HTTP endpoint: POST /api/stripe/webhook File: src/routes/api/stripe/webhook/+server.ts Verifies stripe-signature, constructs Stripe.Event, calls handleStripeEvent.
- Handled event types (business logic): checkout.session.completed; customer.subscription.created / updated / deleted; invoice.paid; invoice.payment_failed; unhandled types are logged and deduped. File: src/lib/server/stripe/webhook-dispatch.ts
- Structured server logs use console.error prefix [stripe_webhook] with stage, eventId, and kind — visible in hosting logs or local dev terminal, not committed to the repo.

Operational verification (for humans / Claude Code in a live session):

- stripe listen --forward-to localhost:5173/api/stripe/webhook
- Trigger checkout or stripe trigger
- Expect HTTP 200 from webhook route and updated data/stripe-billing.json and data/stripe-webhook-events.json (and terminal output from Stripe CLI).

5. GOOGLE ANALYTICS SETUP — GTM / GA4 dashboard showing recent activity

VERDICT: CODE INTEGRATION PASS — DASHBOARD ACTIVITY NOT VERIFIABLE FROM REPO

Evidence:

- GTM bootstrap, dataLayer, Consent Mode v2, page_view on afterNavigate, identifyUser payload, login/sign_up events. Files include: src/routes/+layout.svelte, src/lib/analytics/dataLayer.ts, src/lib/components/tracking/GoogleTagManager.svelte, src/lib/components/tracking/ConsentHead.svelte, src/lib/analytics/consentHeadScript.ts, src/lib/analytics/klaroConfig.ts
- Container id: environment variable PUBLIC_GTM_CONTAINER_ID (GTM-...). File: src/routes/+layout.server.ts
- GA4 tags themselves are configured inside the GTM web UI (not in this repository).

Operational verification:

- Set PUBLIC_GTM_CONTAINER_ID, publish GTM container with GA4 tag, use GTM Preview or GA4 Realtime reports while browsing the app.

6. HOTJAR INTEGRATION — Hotjar dashboard with at least one connected site

VERDICT: CODE / CONSENT HOOKS PASS — HOTJAR SITE CONNECTION NOT VERIFIABLE

Evidence:

- Klaro service and dataLayer event klaro-hotjar-accepted for gating (non-en locales). Password fields use data-hj-suppress on login, signup, and account password change forms. Files: src/lib/analytics/klaroConfig.ts, src/routes/login/+page.svelte, src/routes/signup/+page.svelte, src/routes/account/+page.svelte
- Hotjar script loading is expected to be done via GTM (per project docs), not hardcoded in repo.

Operational verification:

- In Hotjar account, confirm site URL, verify recordings/reports after accepting cookies (non-en) or with Consent granted (en).

SUMMARY TABLE

Criterion Codebase Live / external

Pricing tiers + checkout redirect Pass Configure env + test Payment success UX Partial Add explicit success UI? Billing fields + webhook dedupe Pass (JSON) Confirm RDBMS if required Stripe webhook + CLI logs Implemented stripe listen + triggers GTM / GA4 activity Implemented GTM + GA4 UI Hotjar connected site Partial (code) Hotjar UI + GTM tag

END OF REVIEW

Source: docs/REFACTORING–Master–Task–List.md

REFACTORING — Master Task List & Repo Audit Prompt

Refactoring keeps the codebase scalable, reduces maintenance cost, and increases velocity by removing hidden complexity before bugs compound.

This document does two things:

- Master task list** — How to catalogue tech debt, prioritize by risk and ROI, and ship refactors safely.

Non-negotiable: commit before you refactor

Make a git commit (on a branch) before starting any refactor work.

If the refactor goes wrong, you can reset or revert. Tiny PRs + a clean baseline make bisection and rollback possible.

Suggested flow:

- git checkout -b refactor/<short-topic>
- git add -A && git commit -m "chore: baseline before refactor <topic>" (or commit current WIP intentionally)
- Implement the smallest vertical slice; commit again.
- Run automated tests + quick manual smoke before merge.

Master task list (catalogue tech debt)

Use this table (copy rows as needed) to track debt. Prioritize by **risk × impact** and **ROI** (time saved, incidents prevented).

ID	Area	Summary	Risk	ROI	Owner	Status	PRs	Tests attached	Rollback / monitor
DEBT-001	...	...	H/M/L	H/M/L	...	backlog	...	unit / e2e / manual	...

Per item, enforce:

- Break work into **small PRs** (one concern per PR).
- Automated tests** (extend existing Vitest/Playwright); **manual smoke** checklist for routes you touched.
- Lint + format** (npm run check , npm run lint when the toolchain is healthy).
- Performance / regression** — note any load that got heavier, bundle imports, or new fetch waterfalls.
- Docs** — one line in PR description: what changed, why, how to verify.
- Recurring** — e.g. monthly “refactor window” to burn down backlog.

Tips

- Attach tests and **ROI notes** to each task.
- Keep PRs **tiny and measurable** (reviewable in < 15–20 minutes when possible).
- Automate **alerts** for regressions (CI failures, error rate, core user flows) where you have hosting.

LLM prompt: full-repository SvelteKit refactor audit

Paste the block below into your AI tool. **Constraint:** the model must **not** propose changing Svelte **runes**, **stores**, or broad **reactivity** architecture. Focus on **routing**, **server/client boundaries**, **providers** (auth, env, data loaders), **validation**, **security**, **caching**, **performance**, and **structural duplication**.

```
You are a senior engineer auditing a SvelteKit (Svelte 5) codebase. Read the repository comprehensively: `src/routes`, `src/hooks.server.ts`, `src/lib/server`, `src/lib` (shared client-safe code), `src/app.html`, env handling, API routes, and tests.

### Goals
1. Produce a real summary tied to this repo’s routes, loaders, and patterns—not generic SvelteKit advice.
2. Enumerate issues with evidence: file paths, optional line ranges or short snippets, and why it matters (security, correctness, perf, maintainability).
3. For each issue: concrete fix (minimal diff or canonical SvelteKit pattern), priority (P0 security/correctness → P1 perf/architecture → P2 DX/cleanup), and estimated PR size (S/M/L).
4. Output an exhaustive Refactor Backlog Table suitable for engineering work.

### Out of scope (do not suggest)
- Rewriting state management: no changes to runes usage patterns, global stores, or reactivity model “for cleanliness.”
- Opinion-only style (pure formatting preferences without risk).

### Architectural areas to inspect
- Routing: duplicated or overlapping routes, `+page`/`+layout`/`+server` consistency, `prerender`/`ssr` flags, error boundaries.
- Loaders: `+page.server.ts` / `+layout.server.ts` – validation, early returns, leakage of secrets into `PageData`, unnecessary serializable payload.
- Server/client boundary: `import` of Node-only or server-only modules into client components; `export const ssr` / `csr` misuse; accidental browser-only APIs in server files.
- Providers / auth: session handling, cookie usage, redirects, CSRF-related config on webhooks, trusted origins.
- Env & secrets: `process.env` vs `PUBLIC_*`, validation at startup (`getEnv`), logging that could leak secrets.
- API routes: input validation, rate limits, idempotency, error shapes.
- Caching & data fetching: duplicate fetches, missing cache headers, Strapi or external API patterns.
- Security: CSP hooks, webhook signature verification, HTML injection (`{@html}`), authz on server actions.
- Performance: large client bundles, sync work in `load`, N+1 patterns, heavy dependencies in shared modules.
- Tests & CI: missing coverage on critical paths; flaky e2e.

### Output format (mandatory)

#### A) Executive summary (max 15 bullet points)
Must reference actual route ids or file paths from this repo.

#### B) Refactor Backlog Table
Every row must include:

| ID | Priority | Area | Problem (1–2 sentences) | Evidence (path[:line-range]) | Recommendation | PR size | Tests to add/update |
```

```
Rules:
- Unique ID per row: `REFACT-001`, ...
- No vague rows (forbidden: “improve performance” without a hotspot path and metric).
- If consolidation is proposed, name exact routes or functions to merge and the canonical implementation path.

#### C) Security & correctness appendix
Bullet list: only items with file path + concrete risk (e.g. missing validation on POST body).

#### D) Performance appendix
Bullet list: only items with path + mechanism (blocking `load`, duplicate fetch, bundle concern).

#### E) Migration / rollout
For the top 5 P0/P1 items: ordered steps, rollback (revert PR / feature flag), and what to monitor after deploy.

## Quality bar (“prompt worked” if...)
- Summary is repo-specific (names real routes/modules).
- Backlog table is complete relative to repo size (no hand-waving).
- Every issue maps problem → recommendation with evidence.
- Zero suggestions violate the out-of-scope constraint above.
```

How you know the prompt “worked”

The AI output should:

- Mention **your** routes, loaders, and libs—not only SvelteKit docs links.
- Include a **Refactor Backlog Table** with IDs, paths, line hints, and test notes.
- Avoid vague performance rows; tie hotspots to **files** and **behaviors**.
- Call out **secrets**, **env**, **unsafe imports**, or **auth gaps** with paths when present.
- Respect **no runes / stores / reactivity rewrites** as a hard constraint.

If the model drifts into style-only nits, re-send the **Out of scope** and **Quality bar** sections.

Where this fits (stabilization & scaling)

- **Solo founder / small team** — structured backlog when you lack always-on senior review.
- **AI-assisted development** — recenters boundaries and prevents architectural drift after fast iteration.
- **Learning** — teaches evidence-based backlog, prioritization, and safe incremental delivery.

Optional: attach ROI to each backlog row

Example note field: *“Prevents mis-billed users; saves ~2h/incident debugging webhook drift.”*

Tie ROI to **incident risk**, **support time**, or **onboarding time** for new contributors.

Last updated: template for *nucamp_soloai*. Update the “Paste” block if your stack constraints change.

Source: docs/TS00-Data-TestID-Primer.md

TS00 — Data TestID Primer (start here)

Why: Automated tests need stable hooks into the DOM. Visible labels change with **translations** and **design**; CSS classes change with **Tailwind refactors**. **data-testid** is an explicit contract for “what to click or assert,” independent of look and language.

Rule: Add `data-testid=""` to the **same element the user interacts with** (button, input, link)—not a distant wrapper—using **kebab-case** names (`auth-login-submit` , `pricing-tier-pro`).

Scope: Testing only—never style or select in app CSS via `[data-testid]` .

Tests: Prefer `page.getByTestId('auth-login-submit')` in Playwright instead of `getByText('...')` for critical flows.

Order: Read this primer → apply **TS02-Data-TestID-Setup.md** (full naming map + file priorities) → then Vitest / Playwright docs.

Next: [TS02-Data-TestID-Setup.md](#) · Index: [TS01-Testing-Guide-Index.md](#)

Source: docs/TS01-Testing-Guide-Index.md

TS01 — Testing Implementation Guide (Index / Roadmap)

This pack teaches **Test-Driven Development (TDD)** discipline for this SaaS app: stable selectors first, fast unit tests, realistic end-to-end coverage, and CI-friendly automation—without inventing a parallel testing philosophy.

The AI (or human) implementing tests **must follow the order below**. Skipping steps produces flaky suites (e.g. E2E coupled to translated strings or CSS classes).

Document map

Doc	Purpose
-----	---------

TS01 (this file)	Roadmap, success criteria, LLM rules
TS00 — TS00-Data-TestID-Primer.md	Short “why + rule” for data-testid (read first)
TS02 — TS02-Data-TestID-Setup.md	Full naming conventions, priorities, anti-patterns
TS03 — TS03-Vitest-Setup.md	Unit / integration-style tests for src/lib with Vitest
TS04 — TS04-Playwright-Setup.md	Browser E2E with Playwright
TS05 — TS05-Write-Basic-Tests.md	First priority flows (auth, UI shell, payments prep)

Correct order of operations

1. TS00 — Primer
- Understand why test IDs exist and the one-line rule (kebab-case, on interactive nodes).
2. TS02 — Test IDs (detailed)
- Apply naming patterns and priority file list across the app. Complete before TS05 E2E selectors.
3. TS03 — Vitest
- Cover pure logic, server helpers, serializers, and transforms under `src/lib` (and API helpers) with fast tests. Mock HTTP and secrets; never hit live Stripe/Lemon/Strapi in unit tests unless using recorded fixtures.
4. TS04 — Playwright
- Confirm config (`playwright.config.ts`), base URL, webServer command, and CI assumptions. Add fixtures/page objects as complexity grows.
5. TS05 — Basic E2E + smoke
- Exercise critical paths using **only** `data-testid` or accessibility roles that remain stable across locales.

What “success” looks like

- `npm run test:unit -- --run` passes locally and in CI (when wired).
- `npm run test:e2e` passes against `npm run dev` (or CI webServer).
- Selectors in tests use `getByTestId(...)` or stable **roles + accessible names** where test IDs are inappropriate (e.g. `page <title> checks`).
- Changing **theme classes**, **Paraglide strings**, or **component structure** does not break tests **as long as** behavior and test IDs are preserved.
- New features add tests **with** or **before** implementation when practicing strict TDD.

LLM role: testing engineer, not blind codegen

Across TS02–TS05, the model must:

1. Understand strategy and order
- never generate Playwright suites that depend on visible English copy before TS02 is done.
2. Modify the app for testability
- add `data-testid` per TS02 rules.
3. Align with repo tooling
- Vitest (`vitest.config.ts`), Playwright (`playwright.config.ts` , `e2e/`), npm scripts in `package.json` .
4. Reflect real user behavior
- navigates like a user; avoids implementation-detail selectors.
5. Avoid fragile patterns
- no sleeps for timing; use Playwright auto-wait; no real payment APIs in unit tests.

The model **must not** invent a different stack (Jest-only, Cypress-only, etc.) unless the human explicitly requests migration.

Integration with CI and revenue protection

Automated tests guard **auth**, **billing flows**, and **localized UX**. Extend `.github/workflows/ci.yml` with jobs such as:

- `npm run check`
- `npm run test:unit -- --run`
- `npm run test:e2e` (with Playwright browser install step)

Until CI runs tests on every PR, treat `npm test` as the merge gate locally.

Validation checklist (“did the prompt pack work?”)

- ☐ Team read TS00; key surfaces follow TS02 naming (TS00 + TS02).
- ☐ Unit tests target `src/**/*.{test,spec}.ts` and exclude `e2e/` (TS03).
- ☐ E2E tests live under `e2e/` and use `baseURL` from Playwright config (TS04).
- ☐ Critical flows in TS05 use test IDs, not Tailwind class strings.
- ☐ Tests still pass after **style-only** or **translation-only** changes when IDs and behavior are unchanged.

Related internal docs

- `docs/REFACTORING-Master-Task-List.md` — safe refactors after tests exist.

Repository: `nucamp_soloai` (SvelteKit + Paraglide + Vitest + Playwright).

Source: `docs/TS02-Data-TestID-Setup.md`

TS02 — Data TestID Attribute Setup (conventions & coverage)

Read **TS00-Data-TestID-Primer.md** first for the mental model (why test IDs, one-line rule). **This doc** is the full playbook: naming scheme, where to add attributes first, anti-patterns, and LLM instructions.

Goal: Make the UI stable for automation **without** coupling tests to translated strings, cosmetic CSS classes, or DOM depth.

This is the **foundation** for TS05 Playwright tests and any component tests that query the DOM.

Principles

- 1. **data-testid is for tests only** — never use it for styling (`[data-testid]` in CSS is forbidden).
- 2. **Prefer meaningful, hierarchical names** — reflect feature + role, not implementation.
- 3. **One primary test id per logical control** — button, link, input, card, section root.
- 4. **Paraglide / i18n safe** — labels change per locale; **do not** use `getByText('Sign in')` as the only locator for critical flows once `data-testid` exists.
- 5. **Accessibility still required** — test IDs complement, not replace, `aria-*`, labels, and roles.

Naming convention (required)

Use **kebab-case**, prefixed by area:

```
{area}-{component}-{role}
```

Examples:

Element	Suggested data-testid
Site header root	shell-header
Primary nav	nav-primary
Footer cookie settings button	footer-cookie-settings
Login submit button	auth-login-submit
Login email field	auth-login-email
Login password field	auth-login-password
Signup submit	auth-signup-submit
Pricing tier card (dynamic)	pricing-tier-{tier} e.g. pricing-tier-pro
Pricing subscribe button	pricing-subscribe-{tier}
Account subscription section	account-subscription

For **lists**, include a stable key from data (`id` , `slug` , `tier`), not array index, when possible.

Svelte 5 usage

Add attributes on the actual interactive node:

```
<button type="submit" data-testid="auth-login-submit" class="btn btn-primary">
  {m.login_action_submit()}
</button>

<input
  id="login-email"
  data-testid="auth-login-email"
  type="email"
  ...
/>
```

Avoid putting `data-testid` only on a wrapping `<div>` if the user clicks the `<button>` inside—put the id on the control that receives the action.

Where to apply first (priority order)

- 1. **src/routes/+layout.svelte** — skip link, main landmark ties (optional ids on `Header` / `Footer` children).
- 2. **src/lib/components/Header.svelte** — nav links, auth CTAs.
- 3. **src/lib/components/Footer.svelte** — legal links, cookie settings control.
- 4. **Auth routes** — `login` , `signup` , `forgot-password` forms.
- 5. **pricing/+page.svelte** — plan cards and subscribe buttons.
- 6. **account/+page.svelte** — subscription block, portal buttons, profile forms.

CMS-driven pages should still expose ids on **layout chrome** and **interactive shells** (e.g. FAQ accordion triggers if tested).

Anti-patterns

Do not	Why
<code>page.locator('.btn-primary')</code>	Breaks when design tokens change
<code>getByText('Pricing')</code> alone	Breaks under locale switch
Duplicate ids on one route	Ambiguous locators
Test IDs on every <code></code>	Noise and maintenance

LLM instructions

When editing components:

- 1. Add **only** the attributes needed for planned tests (TS05).
- 2. Keep names **consistent** with the table patterns above.
- 3. Document new ids in the PR description (`Added data-testid ... for E2E`).
- 4. After adding ids, **update or add** Playwright tests to use `page.getByTestId('...')` .

Verification

- Run TS05 tests (or a temporary Playwright spec) using **only** test IDs for those elements.
- Switch Paraglide locale in the app manually; the same test IDs should remain in the DOM.

Previous: `TS00-Data-TestID-Primer.md` — Next: `TS03-Vitest-Setup.md`

Source: `docs/TS03-Vitest-Setup.md`

TS03 — Vitest Unit Testing Setup

Goal: Run **fast, deterministic** tests for logic under `src/lib` (especially server-side helpers) without booting a browser or calling live paid APIs.

This repo already uses **Vitest** with a `$lib` alias. The AI must extend—not replace—that setup.

Current configuration (nucamp_soloai)

- **Config:** `vitest.config.ts`
 - `$lib` → `src/lib`
 - **Include:** `src/**/*.{test,spec}.{js,ts}`
 - **Exclude:** `e2e` , `node_modules` , `dist` , `.svelte-kit`
- **Scripts:** `package.json`
 - `npm run test:unit` — Vitest watch mode
 - `npm run test` — `unit` (`--run`) then Playwright

What to unit test

Test	Examples in repo
Pure functions	Tier mapping, normalization, comparison helpers
Parsing / validation	Webhook signatures, env-dependent guards via injected deps
Idempotent logic	Dedupe keys, status mapping
Small modules with mocks	Stripe/Lemon dispatch when Stripe SDK is mocked

Prefer not to unit test (without extra tooling)

- Full Svelte components (use Playwright or add `@testing-library/svelte` only if the course requires it).
- Full SvelteKit `load` functions in isolation—prefer testing extracted pure helpers or E2E.

Mocking rules

- 1. **No live network** in unit tests for Stripe, Lemon, Strapi, Mautic, OpenAI.
- 2. Mock `fetch` , or pass **injectable** clients into functions under test.
- 3. **Secrets:** never import real `.env` ; use `vi.stubEnv` sparingly and reset in `afterEach` .
- 4. **Filesystem:** billing JSON under `data/` —tests should use temp dirs or mock `fs` if testing persistence (today most tests target pure logic).

File placement

- Colocate: `src/lib/server/foo/bar.ts` → `src/lib/server/foo/bar.test.ts`
- Or sibling `*.spec.ts` —both match `vitest.config.ts` include pattern.

Existing examples: `src/lib/server/**/*.test.ts` (Stripe, Lemon, Mautic, billing).

Coverage (optional enhancement)

Coverage is **not** configured by default. To add:

```
npm i -D @vitest/coverage-v8
```

Then extend `vitest.config.ts` with `coverage: { provider: 'v8', reporter: ['text', 'html'] }` and a `test:coverage` script. Treat coverage as a **signal**, not a target to game.

CI recommendation

Add to `.github/workflows/ci.yml` (alongside existing jobs):

```
- run: npm ci
- run: npm run check
- run: npm run test:unit -- --run
```

Use `NODE_ENV: test` and minimal env vars so `validate-env` and app bootstrap succeed.

LLM instructions

- 1. **Read** `vitest.config.ts` before proposing changes.
- 2. **Mirror** existing test style (Vitest `describe / it`, TypeScript).
- 3. **Add tests** when fixing bugs (regression first) or when TS05 identifies untested pure logic.
- 4. **Do not** duplicate E2E scenarios in Vitest—keep unit tests small.

Commands reference

```
npm run test:unit           # watch
npx vitest run              # single run (same as npm run test:unit -- --run)
```

Previous: `TS02-Data-TestID-Setup.md` — Next: `TS04-Playwright-Setup.md`

Source: `docs/TS04-Playwright-Setup.md`

TS04 — Playwright E2E Testing Setup

Goal: Verify the app **as a user** in a real browser—routing, auth redirects, i18n, and critical flows—while keeping suites **parallel-safe** and **CI-ready**.

Current configuration (nucamp_soloai)

- **Config:** `playwright.config.ts`
 - `testDir`: `e2e`
 - `baseUrl`: `http://127.0.0.1:5173`
 - `webServer`: starts `npm run dev -- --host 127.0.0.1 --port 5173 --strictPort`
 - `reuseExistingServer`: `!process.env.CI` (local reuse OK)
 - `Retries`: `2` on CI, `0` locally
 - `Workers`: `1` on CI for stability
 - `Trace`: `on-first-retry`
 - `Project`: Desktop Chrome (`channel: 'chrome'`)
- **Scripts:** `npm run test:e2e → playwright test`
- **Existing specs:**
 - `e2e/routes.spec.ts` — public routes, titles, `/account` redirect when signed out
 - `e2e/pg03-language-switcher.spec.ts` — locale behavior

Installation / system deps

CI must install browsers:

```
- run: npx playwright install --with-deps chromium
```

(Adjust if you drop `channel: 'chrome'` and use bundled Chromium only.)

Patterns to adopt

Page objects (recommended as specs grow)

Place under `e2e/pages/` or `e2e/fixtures/` :


```
// e2e/pages/login.page.ts
import type { Page } from '@playwright/test';

export function loginPage(page: Page) {
  return {
    goto: () => page.goto('/login'),
    email: () => page.getByTestId('auth-login-email'),
    password: () => page.getByTestId('auth-login-password'),
    submit: () => page.getByTestId('auth-login-submit')
  };
}
```

Use **after** TS02 adds those test IDs.

Fixtures for authenticated sessions

For logged-in flows, prefer:

- **Storage state:** login once in `globalSetup`, save `storageState`, reuse in projects; or
- **Short-lived test user** created via API route / seed script (document secrets handling).

Do **not** commit real passwords or production URLs.

Stability

- Avoid `page.waitForTimeout()` except when debugging.
- Use `expect(locator).toBeVisible()` and Playwright `auto-wait`.
- For multilingual routes, assert `data-testid` or `toHaveURL` patterns, not brittle copy.

Environment variables

E2E runs against local dev server by default. If tests need **test doubles**:

- Stripe Checkout: use **Stripe test mode** keys only in isolated staging; or mock at network layer (`page.route`) for UI-only checks.
- Never point CI at production payment endpoints.

CI recommendation

Extend `.github/workflows/ci.yml`:

```
playwright:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: '20'
        cache: npm
    - run: npm ci
    - run: npx playwright install --with-deps chromium
    - run: npm run test:e2e
  env:
    CI: true
```

Ensure `validate-env` / dev dependencies allow the dev server to boot (provide minimal `PUBLIC_*` and auth paths if required).

LLM instructions

1. **Do not** change `baseURL` or `webServer` without updating docs and CI.
2. **Prefer** `getByTestId` from TS02 for flow tests.
3. **Keep specs focused**—one scenario per `test()` when possible.
4. **Tag** slow tests (`test.slow()` or project tags) if checkout flows are added later.

Commands reference

```
npm run test:e2e
npx playwright test e2e/routes.spec.ts
npx playwright show-report # if HTML reporter added later
```

Previous: `TS03-Vitest-Setup.md` — Next: `TS05-Write-Basic-Tests.md`

Source: `docs/TS05-Write-Basic-Tests.md`

TS05 — Write Basic Tests (Auth, UI, Payments preparation)

Goal: Turn setup (TS02–TS04) into **real coverage** on the flows that protect **revenue and trust**: authentication shell, navigation, pricing/checkout entry points, and account visibility—**without** flaky coupling to layout CSS or translations.

Preconditions

- ☐ TS00 understood; TS02 `data-testid` attributes exist for elements you assert (auth form, pricing buttons, account sections).
- ☐ `npm run test:unit -- --run passes`.
- ☐ `npm run test:e2e` passes with current specs.

Priority matrix (build in this order)

Priority	Flow	Unit (Vitest)	E2E (Playwright)
P0	Signed-out user cannot access <code>/account</code>	—	Already partially covered in <code>e2e/routes.spec.ts</code>
P0	Login form validates presence / error states	Optional helpers	Submit invalid credentials → error UI via test IDs
P1	Signup happy path (test DB / ephemeral user)	—	Requires TS02 ids + test env
P1	Pricing page shows plans when env configured	Pricing helpers	Visible cards / subscribe buttons by test ID
P2	Checkout redirect starts (Stripe test mode or mocked route)	Mock POST handlers	<code>page.route</code> or skip if no keys in CI
P2	Account page shows subscription section when billing JSON seeded	billing-store helpers	Optional with fixture user

Auth tests (patterns)

E2E — login failure (no secrets committed)

```
import { expect, test } from '@playwright/test';

test('login shows error for invalid credentials', async ({ page }) => {
  await page.goto('/login');
  await page.getByTestId('auth-login-email').fill('nobody@example.com');
  await page.getByTestId('auth-login-password').fill('wrong-password');
  await page.getByTestId('auth-login-submit').click();
  await expect(page.getByRole('alert')).toBeVisible();
});
```

E2E — redirect preservation

Already aligned with `redirectTo` query; extend with:

```
await page.goto('/pricing');
// click login from pricing-specific CTA when test ID exists
// expect URL to contain redirectTo after login attempt
```

UI / layout tests

- Header: logo/home link, auth state (sign in vs dashboard)—use **test IDs** on header controls.
- Footer: legal links navigate—can keep role-based locators **if** link text is stable **or** add `data-testid` per TS02.
- Language switcher: extend `e2e/pg03-language-switcher.spec.ts` ; after locale change, **re-query** by test ID, not previous locale string.

Payments (realistic scope)

Full Stripe Checkout completion in E2E is optional and environment-heavy. Recommended incremental approach:

- Unit:** webhook dedupe, tier mapping, provider resolution (`paymentProcessorForLocale`).
- E2E level A:** Pricing page loads; subscribe buttons **disabled** or **enabled** based on fixture env—assert **navigation intent** (button visible + test ID).
- E2E level B:** Intercept `POST **/api/stripe/checkout` with `page.route` and return `{ url: 'https://example.com fake' }` to assert client handles redirect **without** leaving CI network—document as contract test.
- Manual / staging:** complete once with Stripe test keys.

Never assert production card charges in automated CI without isolated test accounts.

Fragile patterns to avoid

Pattern	Instead
<code>expect(page.getByText('Subscribe')).toBeVisible()</code>	<code>getByTestId('pricing-subscribe-pro')</code>
Fixed <code>waitForTimeout(3000)</code>	<code>expect(...).toBeVisible()</code> / expect navigation
One giant test	Arrange ** Arrange login ** / pricing / account in separate <code>test()</code> blocks
Shared browser state without isolation	Independent contexts or serial tests only when documented

TDD workflow reminder

- 1. **Red**: write failing Playwright or Vitest test describing desired behavior.
- 2. **Green**: minimal implementation (or add `data-testid` only).
- 3. **Refactor**: clean up with tests still green.

For bugs: **regression test first**, then fix.

Definition of done (TS05)

- ☐ Auth routes use **test IDs** in E2E assertions.
- ☐ Pricing critical CTAs use **test IDs**.
- ☐ No E2E depends solely on English visible strings for P0/P1 flows.
- ☐ New Vitest tests exist for any **pure logic** extracted during payment/auth work.
- ☐ CI job plan documented (even if not merged yet) in TS03/TS04.

LLM instructions

- 1. **Read** existing `e2e/*.spec.ts` before adding duplicates.
- 2. **Extend** incrementally; prefer **new file** `e2e/auth.spec.ts` over bloating `routes.spec.ts` when scenarios multiply.
- 3. **Never** embed API secrets in tests; use env vars from CI secrets.
- 4. When TS02 ids are missing, **add TS02 first** in the same PR as tests.

Previous: [TS04-Playwright-Setup.md](#) — Index: [TS01-Testing-Guide-Index.md](#)

Source: `docs/SA01-Claude-Code-SubAgents.md`

SA01 — Claude Code Sub-Agents (Specialized AI Roles)

By introducing **sub-agents**, you avoid one monolithic assistant juggling UI, payments, security, and deployment in a single thread. Each mini-agent keeps a **focused role**, **clear rules**, and (optionally) **scoped tools**.

What this file does

- 1. Explains **where** agents live and **how** they are formatted.
- 2. Documents **rules** (YAML frontmatter, `model: inherit`, tool defaults).
- 3. Points to **eight ready-made agents** under `.claude/agents/` for this repo.

This is **workflow tooling**, not a runtime app feature. It supports faster iteration, cleaner context, and less “pattern mixing” in a multilingual SaaS with many integrations.

Folder layout

```
.claude/
agents/
  database-prisma.md
  auth-security.md
  ui-component.md
  api-services.md
  payment-systems.md
  i18n-translation.md
  testing.md
  deployment-build.md
```

Commit these files so the whole team (and students) share the same specialist prompts.

Agent file format

Each agent is a **Markdown file** with **YAML frontmatter** at the top, followed by the **system instructions** (what the specialist must prioritize, avoid, and reference in this codebase).

Required frontmatter

Field	Purpose
name	Short slug (matches filename stem).
description	When to invoke this agent (shown in picker / routing).
model: inherit	Use the same model as the parent Claude Code session.

Tools

- **Full access (default)**: omit the `tools` key entirely so the agent can use the normal Claude Code toolset.
- **Read-only specialist**: restrict with:

```
tools: Read, Grep, Glob
```

Use for audits where you want suggestions without auto-edits.

Example skeleton

```
---
name: example-agent
description: Use for concise one-line when-to-use guidance.
model: inherit
---
```

You are ...

The eight agents (summary)

Agent file	Focus
database-prisma.md	Persistence: Prisma when adopted; today also JSON billing + SQLite auth patterns
auth-security.md	Better Auth, sessions, OAuth, webhook CSRF, secrets
ui-component.md	Svelte 5, Tailwind, DaisyUI, accessibility
api-services.md	+server.ts, validation, Strapi/OpenAI integrations
payment-systems.md	Stripe, Lemon Squeezy, webhooks, locale-based checkout
i18n-translation.md	Paraglide, messages/*.json, locale routing
testing.md	Vitest, Playwright, data-testid, docs TS00–TS05
deployment-build.md	CI/CD, build, adapters, env validation

What you ask the LLM to do

When scaffolding or updating agents:

1. **Create** `.claude/agents/` if missing.
2. **Write** all eight `.md` files with valid frontmatter and strong system prompts.
3. Keep prompts **project-specific** (paths like `src/routes` , `src/lib/server` , `docs/TS02–...`).
4. Teach usage: “Use the **testing** agent to add Playwright coverage for login.”

How to verify it worked

Files

- ☐ `.claude/agents/` exists with **8** markdown files (names above).
- ☐ Each file starts with `---` YAML block containing `name` , `description` , `model: inherit` .
- ☐ Full-access agents **omit** `tools:` unless intentionally read-only.

Behavior (sanity prompts)

- **Testing:** “Suggest three stable `data-testid` names for the login form per `docs/TS02` .”
- **deployment-build:** “List commands that should pass before merge (`npm run check` , tests, env validate).”
- **i18n-translation:** “Find user-visible English not routed through Paraglide in `src/routes` .”

Responses should **stay in role** (tests vs UI vs security vocabulary).

Related docs

- `docs/TS01–Testing–Guide–Index.md` — testing workflow for the **testing** agent.
- `docs/REFACTORING–Master–Task–List.md` — evidence-based refactors after tests exist.

Repository: `nucamp_soloai` · Sub-agents are Claude Code–oriented; adapt paths if your IDE uses a different agent format.

Source: `docs/A08–Page–Transitions.md`

A08 — Page Transitions (Route UX)

Goal: Subtle **fade** / **slide** between SvelteKit routes so the SaaS feels polished without hurting performance, accessibility, or integrations (auth, Strapi, Paraglide).

This doc is the **spec** for humans and LLMs. Implementation lives in `src/lib/components/PageTransition.svelte` and `src/routes/+layout.svelte` .

What “good” feels like

- **Duration:** ~200–300ms total perceived transition (implemented ~260ms in / ~200ms out).
- **Motion:** Lightweight **opacity** + **small vertical translate** (`transform` + `opacity` → GPU-friendly).
- **Navigation:** Never **block** routing; avoid long `await` chains before rendering the outlet.
- **Mobile:** Keep work minimal—no layout thrash; `contain: layout style` on the transition shell.

- **Core Web Vitals:** Target $\leq \sim 300\text{ms}$ interaction feel; route transition overhead $< \sim 100\text{ms}$ CPU beyond baseline (measure in Lighthouse when enabled).

Accessibility (required)

1. **prefers-reduced-motion:** `reduce` — animations become **instant** (duration `0`); no forced movement.
2. **Focus** — On **client-side** navigations only, move focus to **#main-content** (`focus({ preventScroll: true })`) so keyboard and SR users land in the main landmark. Skip auto-focus on **first paint** (`afterNavigate` when `from` is missing).
3. **Skip link** — Unchanged; must still jump to `#main-content` and show focus styles (`app.css .skip-link`).
4. **Keyboard-only** — Header/footer links and main content remain tab-order correct; transition wrapper must not use `pointer-events: none` during nav.
5. **Screen readers** — Prefer **focus management** + semantic `<main>` over noisy `aria-live` on every route (avoids duplicate announcements with browser history).

Integration constraints

Area	Requirement
Auth / protected routes	Redirects (<code>/account</code> $\rightarrow$ <code>/login</code>) must not flash protected content; transitions apply only to the rendered outlet after navigation resolves.
Strapi / loading	Page-level loading belongs in <code>route + page.svelte</code> ; transition does not replace skeletons or suspense—avoid “empty flash” by keeping transitions short.
Paraglide / i18n	Route keys use pathname + search , not visible text; no selectors tied to copy.
Analytics	Optional future hook (e.g. <code>transition end</code> $\rightarrow$ <code>dataLayer</code>); not required for v1.

Feature flag & preferences

- **Server flag:** `PUBLIC_PAGE_TRANSITIONS_ENABLED=true` enables transitions for the session payload. Default when unset: **off** (safe for existing deploys).
- **Local override (client):** `localStorage.setItem('pageTransitionsOff', '1')` disables motion **even if** the flag is on (power users / debugging). Remove key or set to `0` to re-enable.
- **Reduced motion** always wins over the flag and `localStorage`.

No new APIs or databases.

Where code plugs in

1. `src/routes/+layout.server.ts` — Exposes `pageTransitionsEnabled` from env.
2. `src/routes/+layout.svelte` — Wraps `{@render children()}` in `PageTransition` when enabled; merges **afterNavigate** for analytics + focus.
3. `src/lib/components/PageTransition.svelte` — `{#key routeKey}` + `fly` transition, duration tied to reduced motion / flag / `localStorage`.
4. `src/app.css` — Optional `.page-transition-root` containment (performance).

Verification checklist

- ☐ Navigate `/` $\rightarrow$ `/pricing` $\rightarrow$ `/features` — consistent subtle fade/slide.
- ☐ Feels fast — no noticeable “stall” before navigation.
- ☐ OS **Reduce motion** on — transitions effectively **disabled** (instant).
- ☐ Logged out $\rightarrow$ `/account` — redirect to login **clean**, no broken transition shell.
- ☐ Slow Strapi page — loading UI still sensible; transition doesn’t look “stuck”.
- ☐ Change locale + navigate — transitions still work; no English-only assumptions.
- ☐ Skip link $\rightarrow$ focus enters main; tab order unchanged.

Future (not v1)

- User setting UI for “Reduce interface motion” synced to `localStorage` .
- View Transitions API where supported, with CSS fallback.
- Analytics event `page_transition_complete` (debounced).

Related: `docs/TS02-Data-TestID-Setup.md` (tests must not depend on transition wrappers).